Foundations of Software Engineering for Derivative Pricing

Alexandre Landi

March 2026

Contents

Preface . . . 4
1 Environment Management . . . 5
1.1 The Problem: Dependency Drift . . . 5
1.2 `pyproject.toml`: The Project Manifest . . . 5
1.3 `uv`: A Modern Package Manager . . . 6
1.3.1 Initialising a project . . . 6
1.3.2 Adding dependencies . . . 6
1.3.3 Running scripts . . . 7
1.3.4 Synchronising an existing project . . . 7
1.4 The Lockfile (`uv.lock`) . . . 7
1.5 Virtual Environments and Isolation . . . 7
1.6 Worked Example: A Complete `uv` Session . . . 8
1.7 Prompt Quality: From Vague to Precise . . . 9
1.7.1 Prompt set 1: Initialising a project . . . 9
1.7.2 Prompt set 2: Configuring dependencies . . . 10
1.7.3 Prompt set 3: Handing the project to a colleague . . . 11
1.8 Questions . . . 12
1.8.1 Open Questions . . . 12
1.8.2 Multiple Choice Questions . . . 12
2 Project Architecture & the Orchestrator Pattern . . . 14
2.1 The Folder Convention . . . 14
2.2 Separation of Concerns . . . 15
2.3 The Orchestrator Pattern . . . 15
2.4 Logging and Auditability . . . 17
2.5 Document Compilation in the Pipeline . . . 17
2.6 Prompt Quality: From Vague to Precise . . . 18
2.6.1 Prompt set 4: Setting up the project structure . . . 18
2.6.2 Prompt set 5: Writing the orchestrator . . . 18
2.6.3 Prompt set 6: Adding logging and auditability . . . 19
2.7 Questions . . . 20
2.7.1 Open Questions . . . 20
2.7.2 Multiple Choice Questions . . . 20
3 Working with AI Code Generators . . . 22
3.1 Problem Formalization . . . 22
3.2 Iterative Refinement . . . 23
3.2.1 Worked example: a prompt-to-code cycle . . . 23
3.3 The Verification Imperative . . . 24
3.4 Manual Verification: “Do One or Two by Hand” . . . 24
3.4.1 Worked example: verifying a bootstrap . . . 25
3.5 Auditing Output with Domain Knowledge . . . 25
3.5.1 Case study: the yield curve rate hierarchy . . . 25
3.6 A Practical Validation Checklist . . . 27
3.7 Prompt Quality: From Vague to Precise . . . 28
3.7.1 Prompt set 7: Bootstrapping a yield curve . . . 28
3.7.2 Prompt set 8: Pricing a structured product . . . 29

3.7.3 Prompt set 9: Adding validation checks . . . 29
3.8 Questions . . . 30
3.8.1 Open Questions . . . 30
3.8.2 Multiple Choice Questions . . . 30
4 Visualization & Professional Reporting . . . 32
4.1 Static Plots with Matplotlib . . . 32
4.2 Interactive Plots with Plotly . . . 33
4.3 Typst for Professional Documents . . . 35
4.4 Excel as Data Source, Not Computation Engine . . . 35
4.5 Prompt Quality: From Vague to Precise . . . 36
4.5.1 Prompt set 10: Creating a publication-quality chart . . . 36
4.5.2 Prompt set 11: Building an interactive visualisation . . . 37
4.5.3 Prompt set 12: Generating an automated report . . . 37
4.6 Questions . . . 38
4.6.1 Open Questions . . . 38
4.6.2 Multiple Choice Questions . . . 38
Conclusion . . . 40
4.7 Project Readiness Checklist . . . 40
4.8 The Engineering Mindset . . . 41
Appendix A — Answer Key . . . 42
Appendix B — Tools & References . . . 43
4.8.1 Recommended Reading . . . 43
4.8.2 References . . . 43

Preface

These notes accompany the *Foundations of Software Engineering for Derivative Pricing* sessions of the Structured Products course. The course is not a derivatives textbook — there are excellent ones already. Instead, it addresses a gap that most curricula leave open: the transition from *understanding* a pricing model to *building a production-quality engine* around it.

The structured product case studies (Energetic PEA, Best Plus 49) serve as a vehicle. They provide a concrete, non-trivial problem — yield curve construction, option pricing, sensitivity analysis — that must be solved under realistic constraints. But the subject of the course is the engineering itself:

- How to configure an environment that any colleague can reproduce.
- How to structure a project so that it can be understood at a glance.
- How to interact productively with generative coding agents.
- How to verify algorithmic output when the algorithm is too complex to read line by line.
- How to produce professional deliverables — charts, reports, presentations — automatically.

The guiding principle is that **the tool is not the product; the process is the product**. A correct price is the minimum expectation. What distinguishes professional work is that it can be reproduced, audited, extended, and presented — by anyone, at any time, without additional instructions.

These notes assume familiarity with Python at an intermediate level and basic knowledge of fixed-income instruments (discount factors, zero-coupon rates, par swap rates). Mathematical derivations are kept to the minimum necessary for verification; the emphasis is on engineering methodology.

The case study materials used throughout — including the Energetic PEA and Step-Up product specifications, the Excel pricing templates, and the volatility surface data — were designed by Johann Barchechath for the Structured Products course (Barchechath, 2026). These materials provide the realistic market environment against which the engineering methods presented here are applied.

Alexandre Landi
Sophia Antipolis, March 2026

1 Environment Management

Learning Objectives

After completing this chapter, you should be able to:

1. Explain why global `pip install` workflows are insufficient for reproducible quantitative projects.
2. Create and configure a `pyproject.toml` manifest for a new Python project.
3. Use `uv init`, `uv add`, `uv sync`, and `uv run` to manage dependencies and execute scripts.
4. Describe the role of the lockfile (`uv.lock`) and explain why it must be committed to version control.
5. Set up an isolated virtual environment and explain its relationship to the lockfile.

A quantitative project that produces correct results on one machine and fails on another is not a finished project — it is a liability. Before writing a single line of pricing logic, the computational environment must be configured so that any analyst, on any machine, can reproduce the exact same outputs. This chapter introduces the tools and conventions that make this possible.

1.1 The Problem: Dependency Drift

Consider a Python project that depends on `scipy` for numerical computation. If analyst A installs `scipy 1.14` and analyst B installs `scipy 1.17`, their results may differ due to changes in numerical algorithms between versions. In a global `pip install` workflow, nothing prevents this divergence.

The consequences are concrete:

- Pricing results that differ by a few basis points between machines.
- Scripts that crash on a colleague's laptop because a package API changed.
- An inability to reproduce results from six months ago because the original dependency versions were never recorded.

The solution is **hermetic environment management**: a system that records the exact set of packages used, isolates them from the rest of the machine, and allows any user to recreate the identical environment with a single command.

1.2 `pyproject.toml`: The Project Manifest

Every Python project begins with a `pyproject.toml` file. This is the manifest that declares the project's identity and its dependencies. Here is the actual file used in the structured products case study:

```
[project]
name = "equity-structured-products"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.14"
dependencies = [
    "matplotlib>=3.10.8",
    "openpyxl>=3.1.5",
    "pillow>=12.1.1",
```

```
    "plotly>=6.6.0",
    "pymupdf>=1.27.2.2",
    "python-docx>=1.2.0",
    "scipy>=1.17.1",
    "xlrd>=2.0.2",
]
```

Each field serves a specific purpose:

Field	Purpose
`name`	A human-readable identifier for the project. Used by packaging tools if the project is ever distributed.
`version`	Semantic version. Increment when changes are significant enough to affect reproducibility.
`requires-python`	The minimum Python version. The environment manager will refuse to install if the interpreter is too old.
`dependencies`	The list of external packages the project needs. Version specifiers like `>=3.10.8` set a minimum; the lockfile (see below) pins the exact version.

Note The `pyproject.toml` declares *what* the project needs. It does not pin exact versions — that is the lockfile's job. This separation allows flexibility (the manifest) with precision (the lock).

1.3 `uv`: A Modern Package Manager

`uv` is a fast, Rust-based package manager that replaces the traditional `pip` + `virtualenv` + `pip-tools` stack with a single tool. Its core commands map to a simple workflow:

1.3.1 Initialising a project

```
uv init
```

This creates a `pyproject.toml` skeleton and a `.python-version` file. It is the first command run when starting any new project.

1.3.2 Adding dependencies

```
uv add scipy matplotlib plotly
```

Each call updates `pyproject.toml` and regenerates the lockfile. Compare this to the traditional workflow:

Traditional (`pip`)	Modern (`uv`)
`python -m venv .venv`	Handled automatically

Traditional (`pip`)	Modern (`uv`)
`source .venv/bin/activate`	Not needed
`pip install scipy`	`uv add scipy`
`pip freeze > requirements.txt`	Automatic lockfile
`pip install -r requirements.txt`	`uv sync`

1.3.3 Running scripts

```
uv run python main.py
```

This single command ensures the virtual environment exists, is synchronised with the lockfile, and then executes the script inside it. There is no separate activation step.

1.3.4 Synchronising an existing project

When a colleague clones the repository, they run:

```
uv sync
```

This reads the lockfile and installs the exact package versions recorded there. The result is a bit-for-bit identical environment.

1.4 The Lockfile (`uv.lock`)

The lockfile is the critical complement to `pyproject.toml`. While the manifest says "I need `scipy >= 1.17`", the lockfile says "I used `scipy` version `1.17.1`, downloaded from this URL, with this SHA-256 hash." It pins every dependency — including transitive dependencies (packages that your packages depend on).

> 💡 **Commit the lockfile**
>
> The `uv.lock` file must be committed to version control. Without it, two analysts running `uv sync` on the same `pyproject.toml` may resolve to different package versions, defeating the purpose of environment management.

The lockfile also records the Python version, the platform, and the resolution strategy. This means the environment is reproducible not just across machines, but across time: running `uv sync` six months from now will install the same packages, even if newer versions have been released.

1.5 Virtual Environments and Isolation

When `uv sync` runs, it creates a `.venv/` directory inside the project folder. This directory contains a complete, self-contained Python installation with all declared dependencies. Packages installed here do not interfere with other projects or with the system Python.

```
equity-structured-products/
├── .venv/          ← isolated environment
├── data/
├── scripts/
├── pyproject.toml
└── uv.lock
```

If you delete `.venv/` entirely, running `uv sync` will recreate it in seconds. The lockfile is the source of truth — the virtual environment is disposable.

> **Note** Never install packages globally with `pip install <package>`. Global installations create invisible dependencies that cannot be tracked or reproduced. Every package should be declared in `pyproject.toml` and installed via `uv`.

1.6 Worked Example: A Complete `uv` Session

The following terminal transcript shows the full lifecycle of setting up a new quantitative project from an empty directory.

Step 1 — Initialise the project.

```
$ uv init
Initialized project `my-project`
$ ls
pyproject.toml  .python-version  hello.py
```

`uv init` creates a minimal `pyproject.toml`, a `.python-version` file recording the interpreter version, and a placeholder script.

Step 2 — Add the required packages.

```
$ uv add scipy matplotlib xlrd
Resolved 12 packages in 340ms
Installed 12 packages in 45ms
 + matplotlib==3.10.8
 + scipy==1.17.1
 + xlrd==2.0.2
 + ... (transitive dependencies)
```

Each `uv add` call updates `pyproject.toml` (adding the package to the `dependencies` list), resolves the full dependency tree, and writes the result to `uv.lock`. The packages are installed into `.venv/` automatically.

Step 3 — Verify the environment.

```
$ uv run python -c "import scipy; print(scipy.__version__)"
1.17.1
```

`uv run` ensures the virtual environment is synchronised, then executes the command inside it. The output confirms that `scipy 1.17.1` is available.

Step 4 — Write and run a script.

```
$ uv run python scripts/step1_curve.py
Pil      T    Rate    ZC_d    ZC_c         DF     DF_chk
-------------------------------------------------------
1D   0.0027  2.7617% 2.8001% 2.7616%   0.999924   0.999924
3M   0.2500  2.9490% 2.9818% 2.9382%   0.992682   0.992682
...
Saved yield_curve.png
```

The script reads market data from `data/`, bootstraps the yield curve, and writes the output chart to `plots/`.

Step 5 — Hand the project to a colleague.

```
$ git add pyproject.toml uv.lock
$ git commit -m "Add dependencies"
$ git push
```

The colleague clones the repository and runs:

```
$ uv sync
Resolved 12 packages in 15ms
Installed 12 packages in 890ms
$ uv run python scripts/step1_curve.py
... (identical output)
```

The lockfile guarantees that the colleague obtains the exact same package versions. The output is reproducible.

Key Takeaways

1. A `pyproject.toml` declares *what* the project needs; the lockfile pins *exactly which versions* were resolved.
2. `uv run` is the single command that guarantees the correct environment before execution.
3. The virtual environment (`.venv/`) is disposable; the lockfile is the source of truth.
4. Never install packages globally — every dependency belongs in `pyproject.toml`.

1.7 Prompt Quality: From Vague to Precise

A recurring theme of this course is that AI code generators are only as good as the instructions they receive. For each significant step in the environment management workflow, we present three prompts of increasing quality. The pattern — *bad*, *average*, *good* — illustrates how precision eliminates ambiguity and prevents errors.

1.7.1 Prompt set 1: Initialising a project

✗ Bad prompt

"Set up my Python project."

The agent receives no context. It does not know which package manager to use, what Python version is required, or whether the project already has source files. It may fall back to `pip` and `venv`, create a `requirements.txt` instead of a `pyproject.toml`, or pick an arbitrary Python version — any of which would violate the reproducibility principles of this chapter.

Average prompt

"I'm starting a new Python project for pricing structured products. Initialise the environment using `uv` and make sure I'm on Python 3.14."

Better: the package manager and the Python version are specified. The agent will run `uv init` and set `requires-python`. But the prompt says nothing about the project name, the directory layout, or what should happen next — so the agent will produce a generic skeleton and may immediately start adding packages or writing code that you did not ask for.

✓ Good prompt

"I'm starting a quantitative project called `equity-structured-products`. Initialise it with `uv` in the current directory, targeting Python >= 3.14. The project will read market data from Excel files and bootstrap yield curves. For now, just set up the environment — create the `pyproject.toml` and `.python-version` files. Don't add any dependencies yet, I'll tell you which ones I need in the next step. Show me the content of `pyproject.toml` when you're done."

This prompt gives the agent the project name, the Python version constraint, a brief description of the project's purpose (which helps the agent make better decisions later), an explicit scope boundary ("don't add dependencies yet"), and a verification request. The agent has no room for assumptions.

1.7.2 Prompt set 2: Configuring dependencies

✗ Bad prompt

"I need scipy."

The agent will likely run `pip install scipy` in the global environment — exactly the anti-pattern this chapter warns against. No version constraint, no lockfile, no isolation.

Average prompt

"Add scipy and matplotlib to my project. I'm using `uv` as my package manager."

The right tool is named, but the prompt omits critical packages (the project also needs `xlrd`, `openpyxl`, and `plotly`). When the student discovers the missing dependencies two hours later, they will need another round-trip with the agent.

✓ Good prompt

> *"Add the following dependencies to my project using `uv add`. I need: `scipy` for numerical computation (interpolation, root-finding), `matplotlib` for static charts, `plotly` for interactive 3D volatility surfaces, `xlrd` for reading the legacy `.xls` pricer files from the trading desk, and `openpyxl` for any `.xlsx` files. After adding them, show me the updated `dependencies` section of `pyproject.toml` and confirm that `uv.lock` was generated."*

Every package is listed with its purpose — this helps the agent verify that nothing is missing and that no package is redundant. The verification step (show `pyproject.toml`, confirm lockfile) closes the loop.

1.7.3 Prompt set 3: Handing the project to a colleague

> ✗ **Bad prompt**
>
> *"Make sure my colleague can run this."*

The agent has no idea what "this" refers to, who the colleague is, what operating system they use, or what "run" means. The output will be vague at best, wrong at worst.

> **Average prompt**
>
> *"Write setup instructions for a colleague. The project uses `uv` and they'll need to run `uv sync` to install everything."*

The right tool and the key command are mentioned, but the prompt does not specify prerequisite software, which files must be committed to Git, or how the colleague can verify that the environment works.

> ✓ **Good prompt**
>
> *"Write a 'Getting Started' section for the README. A colleague who has never seen this project should be able to go from a fresh clone to a working pipeline by following your instructions. Assume they have Python 3.14 but may not have `uv` installed. The steps are: install `uv`, clone the repo, run `uv sync`, then `uv run python main.py`. They should see 'Saved yield_curve.png' in the terminal if everything worked. Mention explicitly that both `pyproject.toml` and `uv.lock` must be in the repository — without the lockfile, reproducibility is not guaranteed."*

This prompt specifies the target audience (someone who has never seen the project), the output format (README section), the exact command sequence, a concrete success criterion, and a critical constraint about version control. A colleague following the generated instructions can reproduce the environment without asking a single question.

> 💡 **The pattern**

Every prompt in this course follows the same progression: (1) give *context* about the project and what you are trying to achieve, (2) specify the *constraints* and *conventions* that apply, (3) define the *expected output* and its format, and (4) include a *verification step* so the agent can confirm success. Prompts that skip any of these elements produce ambiguous results.

1.8 Questions

1.8.1 Open Questions

Question O1.1 — A team of three analysts works on the same pricing project. Analyst A uses Python 3.12, analyst B uses Python 3.14, and analyst C uses Python 3.11. Explain how `uv` and the `pyproject.toml` configuration can prevent version-related discrepancies. What would happen if the `requires-python` field were omitted?

Question O1.2 — Your project depends on `scipy` for numerical optimisation. A new version of `scipy` changes the default tolerance of its root-finding algorithm, causing your discount factors to differ at the sixth decimal place. Describe the sequence of events that would lead to this problem in a workflow *without* a lockfile, and explain how a lockfile prevents it.

Question O1.3 — A colleague argues that creating a virtual environment and a lockfile is "unnecessary overhead for a small project." Write a brief counterargument (3–5 sentences) using a concrete example from this course.

1.8.2 Multiple Choice Questions

Question 1 — A colleague clones your repository and runs `uv sync`. What does this command do?

A. Creates a new `pyproject.toml` from scratch
B. Uploads the project to a package registry
C. Installs the latest versions of all packages from PyPI
D. Reads `uv.lock` and installs the exact package versions recorded there, creating a `.venv/` if needed
E. Deletes the existing virtual environment

Question 2 — What is the relationship between `pyproject.toml` and `uv.lock`?

A. They contain identical information
B. `pyproject.toml` declares minimum version requirements; `uv.lock` pins the exact resolved versions
C. `uv.lock` is generated from `requirements.txt`
D. `pyproject.toml` is auto-generated by `uv lock`
E. Only one of the two should be committed to version control

Question 3 — Why is `uv run python main.py` preferred over `python main.py`?

A. It compiles the script to a binary
B. It uploads execution logs to the cloud
C. It runs the script faster
D. It requires root permissions

E. It automatically ensures the virtual environment exists and is synchronised before executing the script

Question 4 — You delete the `.venv/` directory. What happens?

A. The project is permanently broken
B. Running `uv sync` recreates it from the lockfile in seconds
C. The lockfile is also deleted
D. All source code is lost
E. `pyproject.toml` must be rewritten

Question 5 — An analyst runs `pip install scipy` in their terminal (without `uv`). What is the risk?

A. No risk — this is the recommended workflow
B. `pip` automatically updates `pyproject.toml`
C. The package is installed globally, creating an untracked dependency that may conflict with other projects and cannot be reproduced by colleagues
D. `uv.lock` is updated automatically
E. The installation is blocked by the operating system

2 Project Architecture & the Orchestrator Pattern

Learning Objectives

After completing this chapter, you should be able to:

1. Design a project directory structure that separates input, logic, and output.
2. Explain the contracts governing `data/`, `scripts/`, `plots/`, and `logs/`.
3. Implement an orchestrator (`main.py`) that executes a multi-step pipeline with error handling.
4. Integrate document compilation (Typst) as the final step of an automated pipeline.
5. Produce a traceable audit log of every pipeline run.

A well-structured project reduces cognitive load, prevents errors, and scales gracefully. This chapter introduces the folder conventions and automation patterns used in professional quantitative engineering.

2.1 The Folder Convention

A quantitative project separates concerns into distinct directories. The following structure is used throughout this course:

```
equity-structured-products/
├── data/                  # Immutable source files
│   ├── pricer-energy-3.xls
│   └── ...
├── scripts/               # Modular Python logic
│   ├── step1_curve.py
│   ├── step4_pricing.py
│   ├── step6_sensitivities.py
│   ├── plot_payoff.py
│   └── plot_vol_surface.py
├── plots/                 # Generated visual assets
│   ├── yield_curve.png
│   ├── payoff_energetic.png
│   └── vol_surface.png
├── logs/                  # Execution audit trails
├── docs/                  # Source documents (PDFs, brochures)
├── main.py                # Orchestrator (single entry-point)
├── pyproject.toml
└── uv.lock
```

Each directory has a clear contract:

Directory	Contract
`data/`	Read-only. Contains the original market data files (Excel spreadsheets, CSVs). These files are never modified by scripts — they are the immutable source of truth.

Directory	Contract
`scripts/`	Modular computation. Each script performs one logical step (e.g., bootstrapping, pricing, sensitivity analysis). Scripts read from `data/` and write to `plots/` or `logs/`.
`plots/`	Generated output. Contains charts and figures produced by scripts. These files are disposable — they can always be regenerated by re-running the pipeline.
`logs/`	Audit trails. Contains text logs recording the output of each pipeline run, including timestamps and numerical results.
`docs/`	Reference documents. Marketing brochures, instructions, and other source materials that inform the analysis.

💡 **Immutable data**

The `data/` directory must never be modified programmatically. If a script needs to transform raw data (e.g., stripping a yield curve from an Excel file), the transformed output belongs in `plots/` or `logs/` — not back in `data/`. This guarantees that the original source can always be consulted.

2.2 Separation of Concerns

The directory structure enforces a fundamental engineering principle: *input*, *logic*, and *output* are physically separated.

- **Input** (`data/`, `docs/`): What you received.
- **Logic** (`scripts/`, `main.py`): What you built.
- **Output** (`plots/`, `logs/`): What you produced.

This separation means that deleting every file in `plots/` and `logs/` has zero impact on the project's integrity. Running `uv run python main.py` regenerates all outputs from scratch.

2.3 The Orchestrator Pattern

In a multi-step quantitative pipeline, each script handles one logical phase: one script bootstraps the yield curve, another prices the options, a third runs sensitivities. Running these manually in the correct order is error-prone and untraceable.

The orchestrator pattern consolidates the entire pipeline into a single entry-point: `main.py`.

```
"""
Main Orchestrator – Single Entry-Point.
Automates the full pipeline from data extraction
to professional PDF reporting.
"""

import subprocess
import sys
```

```python
def run_script(script_path, desc):
    """Executes a python script and handles errors."""
    print(f"--- [RUNNING] {desc} ---")
    result = subprocess.run(
        [sys.executable, script_path],
        capture_output=True, text=True,
    )
    if result.returncode != 0:
        print(f"Error: {result.stderr}")
        return False
    print(f"Success: {desc}\n")
    return True

def main():
    scripts = [
        ("scripts/step1_curve.py",
         "Yield Curve Bootstrapping"),
        ("scripts/plot_payoff.py",
         "Payoff Visualization"),
        ("scripts/step4_pricing.py",
         "Option Pricing"),
        ("scripts/step6_sensitivities.py",
         "Sensitivity Analysis"),
    ]

    for script, desc in scripts:
        if not run_script(script, desc):
            sys.exit(1)

    # Compile professional reports
    subprocess.run(
        ["typst", "compile", "presentation.typ"],
        check=True,
    )

if __name__ == "__main__":
    main()
```

The key design decisions are:

1. **Sequential execution with early abort.** If any script fails, the pipeline stops immediately. This prevents downstream scripts from running on stale or missing data.

2. **`subprocess.run` with `capture_output=True`.** Each script runs in its own process. Standard output and standard error are captured, which allows the orchestrator to log results or suppress noise.

3. **`sys.executable`.** Using the current Python interpreter (rather than a hardcoded `python3`) ensures the script runs inside the same virtual environment as the orchestrator.

4. **Report compilation as the final step.** The pipeline ends by calling `typst compile` to generate the client-facing PDF. This means the deliverable is always up-to-date with the latest data.

Note The orchestrator should never contain business logic. It coordinates — it calls scripts in order, checks for errors, and triggers report generation. The actual computation lives in the `scripts/` directory.

2.4 Logging and Auditability

Every pipeline run should produce a traceable record. The simplest approach is to redirect the orchestrator's output to a timestamped log file:

```
uv run python main.py | tee logs/run_2026-03-31.txt
```

This creates a plain-text audit trail that records which scripts ran, whether they succeeded, and what they printed. If a pricing result is questioned weeks later, the log provides evidence of exactly which pipeline produced it.

2.5 Document Compilation in the Pipeline

Modern typesetting systems like Typst can be invoked from the command line, which means they integrate naturally into an automated pipeline.

```
subprocess.run(
    ["typst", "compile", "presentation.typ"],
    check=True,
)
```

This single line converts a Typst source file into a PDF. By placing it at the end of the orchestrator, we guarantee that the client-facing presentation always reflects the latest data and plots.

The alternative — manually opening a document editor, updating figures, and exporting to PDF — is slow, error-prone, and impossible to automate.

💡 **If it cannot run with one command, it is not finished**

A professional quantitative project satisfies a simple test: a colleague who has never seen the code before can clone the repository, run `uv run python main.py`, and obtain every output — data tables, charts, and PDF reports — without any additional instructions.

Key Takeaways

1. Separate *input* (`data/`), *logic* (`scripts/`), and *output* (`plots/`, `logs/`) physically.
2. The orchestrator calls scripts in order, aborts on failure, and compiles reports as the final step.
3. If a colleague cannot run your project with `uv run python main.py`, the project is incomplete.
4. Generated outputs (`plots/`, `logs/`) are disposable — the pipeline recreates them from scratch.

2.6 Prompt Quality: From Vague to Precise

As in Chapter 1, we present three prompts of increasing quality for each significant step. The same pattern applies: *context*, *constraints*, *expected output*, *verification*.

2.6.1 Prompt set 4: Setting up the project structure

✗ Bad prompt

"Organise my files."

The agent does not know which files exist, what the project does, or which conventions to follow. It may rename files arbitrarily, merge directories, or create a structure that conflicts with the orchestrator pattern.

Average prompt

"Create a folder structure for my Python project. I need directories for data, scripts, and output."

The intent is clear, but the prompt omits critical details: should `data/` be read-only? Should output be split into `plots/` and `logs/`? What about documentation? The agent will produce a plausible but generic layout.

✓ Good prompt

"Set up the following directory structure for a structured products pricing project. Create: `data/` for immutable market data files (Excel spreadsheets — these must never be modified by scripts), `scripts/` for modular Python scripts (one script per pipeline step), `plots/` for generated charts, `logs/` for execution audit trails, and `docs/` for reference documents (brochures, specifications). The orchestrator `main.py` goes in the project root. `plots/` and `logs/` are disposable — everything in them can be regenerated by re-running `main.py`. Show me the final tree."

Every directory is named with its contract: what it contains, whether it is read-only or disposable, and how it relates to the pipeline. The verification request ("show me the final tree") lets the student confirm the structure at a glance.

2.6.2 Prompt set 5: Writing the orchestrator

✗ Bad prompt

"Write a main.py that runs my scripts."

Which scripts? In what order? What should happen if one fails? Should it produce logs? Compile a report? The agent will guess — and guesses in pipeline orchestration lead to silent failures or corrupted output.

Average prompt

> *"Write a `main.py` that runs `step1_curve.py`, `step4_pricing.py`, and `step6_sensitivities.py` in order. If any script fails, stop the pipeline."*

The scripts and the sequencing are specified, and the error handling policy is stated. But the prompt does not mention `subprocess.run` vs.
direct imports, `sys.executable` for virtual environment isolation, or report compilation as a final step. The agent may import the scripts directly, breaking process isolation.

> **✓ Good prompt**
>
> *"Write an orchestrator in `main.py` that runs the following scripts sequentially using `subprocess.run`: (1) `scripts/step1_curve.py` — yield curve bootstrapping, (2) `scripts/plot_payoff.py` — payoff visualisation, (3) `scripts/step4_pricing.py` — option pricing, (4) `scripts/step6_sensitivities.py` — sensitivity analysis. Use `sys.executable` as the Python interpreter so the scripts run inside the same virtual environment. Capture stdout and stderr. If any script returns a non-zero exit code, print the error and call `sys.exit(1)` immediately — do not run downstream scripts. As the final step, call `typst compile presentation.typ` to generate the client-facing PDF. The orchestrator should contain zero business logic — only coordination."*

The prompt specifies the execution mechanism (`subprocess.run`), the interpreter (`sys.executable`), the error handling policy (abort on failure), the output capture strategy, the full script list with descriptions, and the final compilation step. It even states the architectural constraint: no business logic in the orchestrator.

2.6.3 Prompt set 6: Adding logging and auditability

> **✗ Bad prompt**
>
> *"Add logging."*

Logging of what? To where? In what format? The agent may add Python's `logging` module with a generic configuration, which is not what the orchestrator pattern requires.

> **Average prompt**
>
> *"Make the pipeline save its output to a log file in the `logs/` directory."*

The target directory is correct, but the prompt does not specify the filename convention (timestamps?), what should be logged (stdout only? stderr?), or whether the log should also appear in the terminal.

> **✓ Good prompt**
>
> *"Modify `main.py` so that every pipeline run produces a log file in `logs/`. The filename should include a timestamp, e.g., `logs/run_2026-03-31_14-30.txt`. The log should capture the full stdout and stderr of every script, including the script name, whether it succeeded or failed,*

> *and the execution time. Also print the output to the terminal in real time (like `tee`). If a pricing result is questioned weeks later, this log should provide enough information to identify exactly which pipeline run produced it."*

The prompt specifies the filename convention, the content (stdout, stderr, status, timing), the dual-output requirement (file + terminal), and — crucially — the *reason* for logging: audit traceability. The agent now understands the purpose, not just the mechanics.

2.7 Questions

2.7.1 Open Questions

Question O2.1 — The orchestrator in `main.py` uses `subprocess.run` to call each script sequentially. Describe two advantages *and* one potential disadvantage of running each script in its own subprocess (as opposed to importing and calling Python functions directly).

Question O2.2 — Consider a project where `plots/` is committed to version control (i.e., generated charts are tracked by Git). Explain why this creates problems for collaboration and describe how the orchestrator pattern makes it unnecessary.

Question O2.3 — A junior analyst inherits a project with no `main.py`. The README says: "Run `step1_curve.py`, then `step4_pricing.py`, then `step6_sensitivities.py`, then open `presentation.pptx` and update the charts manually." Identify the engineering deficiencies in this workflow and propose a remediation plan.

2.7.2 Multiple Choice Questions

Question 6 — In the project structure, the `data/` directory contains:

A. Generated charts and figures
B. Python scripts for pricing
C. Immutable source files (e.g., Excel market data) that are never modified by scripts
D. Compiled PDF reports
E. Virtual environment packages

Question 7 — The orchestrator (`main.py`) calls each script using `subprocess.run([sys.executable, script_path], ...)`. Why `sys.executable` instead of `"python"`?

A. It ensures the script runs inside the same virtual environment as the orchestrator
B. It is faster
C. It enables parallel execution
D. It compiles the script before running it
E. It is required by `uv`

Question 8 — What happens if `step4_pricing.py` fails during a pipeline run?

A. The orchestrator skips it and continues
B. The orchestrator stops immediately (`sys.exit(1)`) to prevent downstream scripts from running on missing data
C. The error is silently ignored
D. The orchestrator retries three times
E. Only the PDF compilation is skipped

Question 9 — You delete the entire `plots/` directory. What is the impact?

A. The project is permanently broken
B. All source data is lost
C. Running `uv run python main.py` regenerates all plots from scratch — the directory is disposable
D. The `pyproject.toml` must be updated
E. The lockfile becomes invalid

Question 10 — The orchestrator calls `typst compile` as its final step. What is the benefit?

A. The client-facing PDF is always up-to-date with the latest data and plots, without manual intervention
B. Typst is faster than Python
C. Typst replaces the need for Python scripts
D. It uploads the PDF to a server
E. It is required by `uv`

3 Working with AI Code Generators

Learning Objectives

After completing this chapter, you should be able to:

1. Write precise, unambiguous prompts that specify data sources, conventions, and expected output formats.
2. Apply the iterative generate–inspect–correct loop to refine AI-generated code.
3. Manually verify the first one or two iterations of an iterative algorithm to audit the loop logic.
4. Use domain knowledge (compounding effects, coupon stripping) to determine whether algorithmic output is correct or buggy.
5. Apply a validation checklist (monotonicity, boundary values, internal consistency) to bootstrapped yield curves.

Generative coding agents can produce functional Python scripts in seconds. This capability does not eliminate the need for engineering skill — it shifts it. The bottleneck is no longer writing syntax; it is *specifying problems precisely* and *verifying outputs rigorously*. This chapter formalises both disciplines.

3.1 Problem Formalization

An AI code generator responds to a prompt. The quality of the generated code is directly proportional to the precision of that prompt. Ambiguous specifications produce ambiguous results.

Consider the task of building a yield curve for a structured product pricing engine. A vague prompt might read:

> *"Build a yield curve from the Excel file."*

This prompt leaves critical questions unanswered:

- Which sheet in the workbook contains the rates?
- What is the day-count convention?
- How far along the curve do we need to go?
- Are the quoted rates simple, discrete, or continuous?
- Should the output include discount factors, zero-coupon rates, or both?

A precise prompt encodes these constraints explicitly:

> *"Read pillar rates from the sheet 'Courbe des taux' in* `pricer-energy-3.xls`*. Rates up to 1Y are money-market (simple interest, Act/360). Rates from 2Y onward are annual par swap rates. We need pillars up to 7 years (product maturity 6Y plus a 1-year buffer for the maturity sensitivity). Output: discrete ZC rates, continuous ZC rates, and discount factors."*

The second prompt eliminates ambiguity. It specifies the data source, the rate conventions, the required horizon, and the expected output format.

💡 **Garbage in, garbage out**

A code generator is a function: it maps an input (your prompt) to an output (code). If the input is vague, the output is unreliable. Investing time in a precise, complete specification is not overhead — it is the primary engineering task.

3.2 Iterative Refinement

In practice, even a well-specified prompt rarely produces perfect code on the first attempt. The effective workflow is a conversation loop:

1. **Generate:** The agent produces a first draft of the script.
2. **Inspect:** Read the code line by line. Does it match the specification?
3. **Test:** Run the script. Does it produce output? Does the output make sense?
4. **Correct:** Identify errors or missing features. Feed them back as a refined prompt.
5. **Document:** The agent should produce documentation (e.g., a README) alongside the code, recording what was built and why.

This loop is not a sign of failure — it is the standard operating procedure. The ability to inspect generated code and identify where it deviates from the specification is the core skill.

3.2.1 Worked example: a prompt-to-code cycle

The following transcript illustrates a realistic interaction with a coding agent for the structured products case study.

Iteration 1 — Initial prompt.

> *"Read the yield curve from `pricer-energy-3.xls`, sheet 'Courbe des taux'. Bootstrap discrete ZC rates and discount factors for pillars up to 7Y."*

The agent produces a script. We run it:

```
$ uv run python scripts/step1_curve.py
KeyError: 'Courbe des Taux'
```

Diagnosis: The sheet name is case-sensitive. The Excel file uses `"Courbe des taux"` (lowercase *t*), but the agent wrote `"Courbe des Taux"` (uppercase *T*).

Iteration 2 — Correction.

> *"The sheet name is case-sensitive. Use 'Courbe des taux' (lowercase t). Also, I notice you are treating all rates as simple interest. Rates beyond 1Y are annual par swap rates — they require iterative stripping."*

The agent corrects the sheet name and implements swap stripping. We run it again:

```
$ uv run python scripts/step1_curve.py
Pil       T    Rate     ZC_d     ZC_c          DF
-----------------------------------------------
```

```
1D   0.0027  2.7617% 2.8001% 2.7616%   0.999924
...
7Y   7.0000  4.7075% 4.7658% 4.6557%   0.721877
```

The script runs. But does the output make sense?

Iteration 3 — Verification.

> *"The output looks plausible. Please also add a validation section: check that DFs are monotonically decreasing, that DF(1D) is close to 1, and plot the three rate series on a single chart."*

The agent adds validation checks and a Matplotlib chart. The final output includes:

```
--- Validation ---
DFs monotonically decreasing: True
DF(1D) = 0.999924 (~ 1.0)
DF(6Y) = 0.761175
Max DF vs exp(-r*T) error: 4.44e-16
Saved yield_curve.png
```

The script is now verified. Three iterations were required — one for a data access error, one for a rate convention error, and one for validation instrumentation. This is typical.

> **Note** The most common errors in AI-generated financial code are not algorithmic — they are **specification errors**: wrong sheet names, wrong rate conventions, wrong column indices. These are exactly the errors that a precise prompt prevents.

3.3 The Verification Imperative

Generated code that compiles and runs without errors is *not* necessarily correct. A script can produce plausible-looking numbers that are entirely wrong — for example, bootstrapping a yield curve with an incorrect day-count convention will produce discount factors that are close to correct but systematically biased.

The burden of proof is always on the engineer, not the tool.

> 💡 **The golden rule**
>
> AI-generated code is guilty until proven innocent. The fact that a script runs without errors proves nothing about the correctness of its output.

3.4 Manual Verification: "Do One or Two by Hand"

The most reliable verification method for iterative algorithms is to manually compute the first one or two iterations and compare them against the script's output.

3.4.1 Worked example: verifying a bootstrap

The AI produces a script that bootstraps 11 discount factors from pillar rates. Rather than trusting the entire output, we manually verify the first non-trivial step: computing the 2-year discount factor from the 1-year discount factor and the 2-year swap rate.

Given:

- 1Y discount factor: $\mathrm{DF}(1) = 0.967863$ (already verified from the money-market rate)
- 2Y par swap rate: $s_2 = 3.8440\%$

Step 1 — the par bond equation. A bond paying annual coupon s_2 at par satisfies:

$$1 = s_2 \cdot \mathrm{DF}(1) + (1 + s_2) \cdot \mathrm{DF}(2)$$

Step 2 — solve for DF(2).

$$\mathrm{DF}(2) = \frac{1 - s_2 \cdot \mathrm{DF}(1)}{1 + s_2}$$

Step 3 — substitute.

$$\mathrm{DF}(2) = \frac{1 - 0.03844 \times 0.967863}{1.03844}$$

$$\mathrm{DF}(2) = \frac{1 - 0.037205}{1.03844} = \frac{0.962795}{1.03844} = 0.927155$$

Step 4 — compare. The script outputs $\mathrm{DF}(2) = 0.927155$. The manual calculation matches to six decimal places.

If the first two iterations match, the loop logic can be trusted for the remaining nine pillars. If they do not match, there is a bug — typically in the rate convention or the indexing of the summation.

> **Note** This method — verifying a small number of iterations by hand — applies to any iterative algorithm: Monte Carlo simulations (check the first few paths), Greeks computation (check one finite difference manually), or portfolio optimisation (verify one weight calculation).

3.5 Auditing Output with Domain Knowledge

Even if the algorithm is correctly implemented, its output must be checked against financial intuition. A correct algorithm applied to the wrong data, or with the wrong parameters, will produce internally consistent but financially meaningless results.

3.5.1 Case study: the yield curve rate hierarchy

After bootstrapping, the pricing engine produces three series for each pillar: the raw pillar rate (par swap rate), the discrete zero-coupon (ZC) rate, and the continuous ZC rate. Inspecting the output table for maturities beyond one year reveals a consistent ordering:

$$r_{\text{continuous}} < r_{\text{pillar}} < r_{\text{discrete ZC}}$$

Is this ordering correct, or is it a sign of a bug?

3.5.1.1 Why $r_{\text{continuous}} < r_{\text{discrete}}$

This is the **compounding frequency effect**. In discrete annual compounding, interest is calculated once per year and added to the principal. In continuous compounding, interest is calculated and reinvested over an infinitely fine subdivision of time.

Because the continuously compounded interest generates its own interest without pause, a *lower* quoted rate is sufficient to produce the same terminal wealth. The mathematical relationship is:

$$r_{\text{cont}} = \ln(1 + r_{\text{disc}})$$

Since $\ln(1 + x) < x$ for all $x > 0$, the continuous rate is always strictly below the discrete rate.

3.5.1.2 Why $r_{\text{discrete ZC}} > r_{\text{pillar}}$

This is the **coupon stripping effect**. The pillar rate for maturities beyond one year is a par swap rate: the coupon rate at which a coupon-bearing bond trades at par. A par bond pays intermediate coupons that the investor can reinvest.

A zero-coupon bond, by contrast, defers all cash flows to maturity. The investor receives no intermediate payments and therefore cannot reinvest. In an upward-sloping yield curve environment, this deferred payment must be compensated by a *higher* terminal rate.

In other words, stripping removes the "averaging" effect of intermediate coupons, revealing the true term rate — which is higher than the blended par rate.

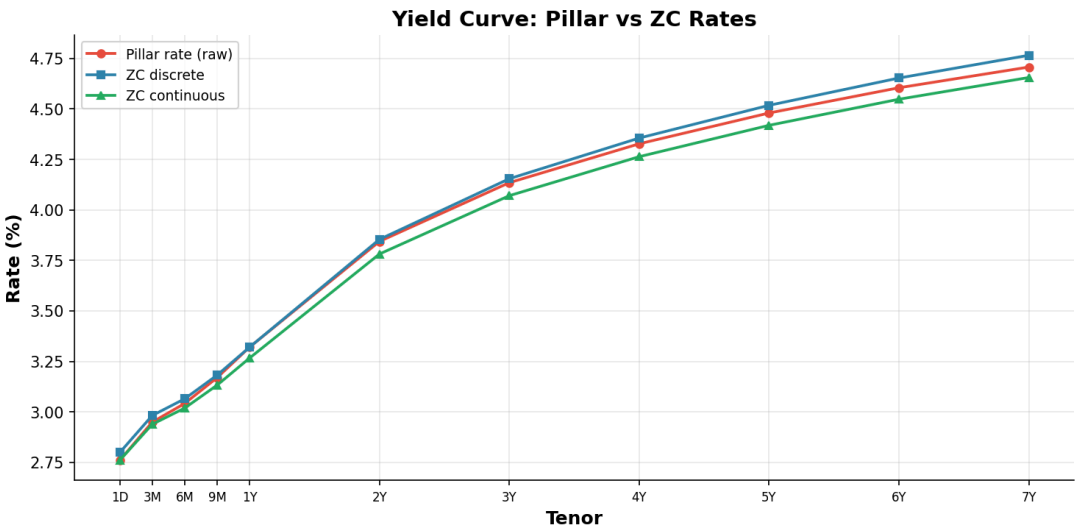

Figure 1: Yield curve output from the bootstrapping pipeline. Three series are plotted: the raw pillar rates (par swap rates, red), discrete zero-coupon rates (blue), and continuous zero-coupon rates (green). The ordering $r_{\text{cont}} < r_{\text{pillar}} < r_{\text{disc ZC}}$ is visible at the long end and is explained by the compounding frequency effect and the coupon stripping effect.

> 💡 **Verification through financial theory**
>
> Without understanding the compounding frequency effect and the coupon stripping effect, an engineer cannot determine whether the ordering $r_{\text{cont}} < r_{\text{pillar}} < r_{\text{disc ZC}}$ is correct or a bug. Domain knowledge is the only tool capable of auditing algorithmic output at this level. Figure 1 illustrates the output that must be audited.

3.6 A Practical Validation Checklist

After verifying the algorithm manually and auditing the output against financial theory, apply these mechanical checks:

Check	**Expected result**
Discount factors monotonically decreasing	$\mathrm{DF}(T_1) > \mathrm{DF}(T_2)$ for $T_1 < T_2$ — a dollar tomorrow is worth more than a dollar next year.
DF(overnight) ≈ 1.0	The 1-day discount factor should be very close to 1 (e.g., 0.999924).
Internal consistency	$\mathrm{DF} = (1 + r_{\text{disc}})^{-T}$ should equal $\exp(-r_{\text{cont}} \cdot T)$ to machine precision.
Boundary behaviour	At $T = 0$, DF $= 1$ exactly. As $T \to \infty$, DF $\to 0$.

For reference, the full output of the bootstrapping pipeline is reproduced below. Each row can be audited against the checks above.

Pillar	**T (y)**	**Rate**	**ZC disc**	**ZC cont**	**DF**
1D	0.003	2.762%	2.800%	2.762%	0.999924
3M	0.250	2.949%	2.982%	2.938%	0.992682
6M	0.500	3.041%	3.064%	3.018%	0.985021
9M	0.750	3.169%	3.182%	3.132%	0.976782
1Y	1.000	3.320%	3.320%	3.266%	0.967863
2Y	2.000	3.844%	3.854%	3.782%	0.927155
3Y	3.000	4.135%	4.155%	4.071%	0.885044
4Y	4.000	4.328%	4.356%	4.264%	0.843203
5Y	5.000	4.480%	4.518%	4.419%	0.801759
6Y	6.000	4.605%	4.653%	4.548%	0.761175
7Y	7.000	4.708%	4.766%	4.656%	0.721877

Table 1: Full bootstrap output. The raw pillar rates (column 3) are money-market rates for $T \leq$ 1Y and par swap rates for $T >$ 1Y. The ordering $r_{\text{cont}} < r_{\text{pillar}} < r_{\text{disc ZC}}$ is visible in every row beyond 1Y. Discount factors decrease monotonically from 0.9999 (overnight) to 0.7219 (7Y).

Key Takeaways

1. AI-generated code is guilty until proven innocent — running without errors proves nothing about correctness.
2. Precise prompts prevent the most common errors: wrong sheet names, wrong rate conventions, wrong column indices.

3. Manual verification of one or two iterations validates the loop logic for the entire algorithm.
4. The ordering $r_{\text{cont}} < r_{\text{pillar}} < r_{\text{disc ZC}}$ is correct and explained by the compounding frequency effect and the coupon stripping effect.

3.7 Prompt Quality: From Vague to Precise

Chapter 3 is where prompt quality matters most. The financial domain is full of implicit conventions — day-count fractions, compounding frequencies, rate types — that an AI agent will not guess correctly unless they are stated explicitly.

3.7.1 Prompt set 7: Bootstrapping a yield curve

✗ Bad prompt

"Build a yield curve from the Excel file."

Which Excel file? Which sheet? What kind of rates are quoted? What day-count convention? How many pillars? The agent will make assumptions about every one of these — and at least one assumption will be wrong.

Average prompt

"Read the yield curve from `pricer-energy-3.xls`, sheet 'Courbe des taux'. Bootstrap ZC rates and discount factors for all available pillars."

The data source and sheet name are correct. But the prompt does not distinguish between money-market rates (simple interest, Act/360) and par swap rates (annual compounding). The agent will likely apply the same formula to all pillars, producing systematically biased discount factors beyond 1Y that look plausible but are wrong.

✓ Good prompt

"Read pillar rates from `pricer-energy-3.xls`, sheet 'Courbe des taux' (case-sensitive, lowercase t). Rates up to and including 1Y are money-market rates: simple interest, Act/360 day-count convention. Rates from 2Y onward are annual par swap rates — bootstrap them using iterative coupon stripping. I need pillars from 1D to 7Y (the product matures at 6Y, plus a 1-year buffer for the maturity sensitivity). For each pillar, output: the raw rate, the discrete ZC rate, the continuous ZC rate, and the discount factor. Save the script as `scripts/step1_curve.py` and write the output both to the terminal and to a chart (`plots/yield_curve.png`) with all three rate series."

Every financial convention is spelled out: the day-count fraction, the compounding type for each tenor bucket, the stripping method, the pillar range with its justification. The output format is fully specified. The agent has enough information to produce a correct bootstrap on the first attempt.

3.7.2 Prompt set 8: Pricing a structured product

✗ Bad prompt

"Price the structured product."

Which product? Using which model? With what market data? What is the expected output — a single price, a table, a report? The agent has nothing to work with.

Average prompt

"Price the Energetic PEA product using the yield curve we just built. It's a capital-guaranteed product with participation in the DJ Euro Stoxx 50."

The product name and underlying are mentioned, but the prompt omits the payoff formula, the barrier levels, the cap, the bonus mechanism, the maturity, and the volatility source. The agent will need to guess the payoff structure — and structured products have too many variants for guessing to work.

✓ Good prompt

"Price the Energetic PEA product. Here are the exact specifications: 6-year maturity, capital guaranteed at 100% of nominal. Payoff at maturity: if the DJ Euro Stoxx 50 performance is between 0% and +50%, the investor receives 100 + performance. If performance exceeds +50%, a bonus of 10 percentage points is added, so the investor receives 100 + performance + 10. The payoff is capped at 160. If performance is negative, the investor receives 100 (capital guarantee). Use the discount factors from `step1_curve.py` *and the implied volatility surface from the sheet 'Vol implicite' in* `pricer-energy-3.xls`*. Price using Black-Scholes with a flat vol interpolated at the 6Y maturity. Output the theoretical price and the separate values of each component (zero-coupon bond + call spread + digital)."*

The payoff is described precisely with all thresholds. The data sources are named. The pricing model is specified. The output includes component decomposition, which allows the student to verify each piece independently — a crucial verification strategy for structured products.

3.7.3 Prompt set 9: Adding validation checks

✗ Bad prompt

"Check if the results are correct."

Correct against what? The agent has no benchmark, no expected values, no validation criteria. It may add a trivial check (`assert price > 0`) that proves nothing.

Average prompt

"Add some validation to the yield curve script. Make sure the discount factors look reasonable."

"Look reasonable" is subjective. The agent will add a few `print` statements but will not implement systematic checks.

✓ Good prompt

"Add a validation section at the end of `step1_curve.py`. Check the following: (1) discount factors are monotonically decreasing — DF(T1) > DF(T2) whenever T1 < T2; (2) DF(1D) is close to 1.0 (within 0.001); (3) internal consistency: for each pillar, the DF computed from the discrete rate must equal the DF computed from the continuous rate to machine precision (error below 1e-14); (4) print a clear PASS/FAIL for each check. If any check fails, exit with a non-zero code so the orchestrator catches it."

Each validation criterion is mathematically precise, with explicit tolerances. The exit code requirement ensures that validation failures propagate through the orchestrator — a failed check stops the pipeline, exactly as it should.

3.8 Questions

3.8.1 Open Questions

Question O3.1 — You ask an AI coding agent to "price a structured product." The agent produces a 200-line script that runs without errors and outputs a price of 98.45. Describe, in order, the steps you would take to determine whether this price is trustworthy. Your answer should reference at least two verification methods discussed in this chapter.

Question O3.2 — The yield curve table (Table 1) shows that for maturities up to 1 year, the ZC discrete rate and the pillar rate are *very close* but not identical. For maturities beyond 1 year, the gap widens significantly. Explain why this is the case, referring to the difference in rate conventions between the short end and the long end of the curve.

Question O3.3 — An AI agent produces a Monte Carlo pricer for an exotic option. The script generates 100,000 paths and returns a price. You cannot verify 100,000 paths by hand. Describe a practical strategy for auditing the correctness of this script, using the "manual verification" principle adapted to the Monte Carlo setting.

3.8.2 Multiple Choice Questions

Question 11 — A code generator produces a yield curve bootstrapping script. The script runs without errors. Which statement is correct?

A. If the script runs, the output is correct
B. Code generators never produce bugs in numerical code
C. Running the script twice is sufficient verification
D. The output should be verified by manually computing one or two discount factors by hand
E. Verification is only needed for financial applications

Question 12 — You manually compute $\mathrm{DF}(2) = 0.927155$ and the script outputs $\mathrm{DF}(2) = 0.927155$. What can you conclude?

A. The entire script is provably correct
B. No further verification is needed
C. The loop logic for the first non-trivial iteration matches the manual calculation; the algorithm can be trusted for subsequent iterations

D. The manual calculation must be wrong
E. The script used a different formula

Question 13 — The bootstrapped yield curve shows $r_{\text{discrete ZC}} > r_{\text{pillar}}$ for all maturities beyond 1 year. This is:

A. A bug — ZC rates should be lower than par rates
B. A rounding error
C. Caused by using the wrong day-count convention
D. Correct — the coupon stripping effect causes ZC rates to exceed par rates when the curve is upward-sloping
E. Only true for government bonds

Question 14 — The lockfile records that `scipy==1.17.1` was used. Six months later, `scipy 1.18` is released with changes to the optimisation algorithms. What happens when a colleague runs `uv sync`?

A. `scipy 1.18` is installed automatically
B. `scipy 1.17.1` is installed, because the lockfile pins the exact version
C. The lockfile is updated to `1.18`
D. `uv sync` fails because `1.17.1` is no longer available
E. The colleague must manually edit the lockfile

Question 15 — A vague prompt produces code that runs but uses an incorrect day-count convention. The resulting discount factors are:

A. Identical to the correct values
B. Correct, because the convention does not matter
C. Obviously wrong (e.g., negative discount factors)
D. Caught by Python's type checker
E. Close to correct but systematically biased — the error is invisible without domain knowledge

4 Visualization & Professional Reporting

Learning Objectives

After completing this chapter, you should be able to:

1. Produce publication-quality static charts with Matplotlib, including proper axis labels, legends, and high-resolution output.
2. Generate interactive 3D visualisations (e.g., volatility surfaces) with Plotly and export them as self-contained HTML files.
3. Integrate document typesetting (Typst) into the automated pipeline so that reports are always current.
4. Articulate why Excel should serve as a data source rather than a computation engine in professional workflows.

Quantitative results are only as useful as their presentation. Raw numerical output — a table of discount factors, a list of option prices — is necessary for verification but insufficient for communication. This chapter covers the tools and practices for transforming computational results into professional deliverables.

4.1 Static Plots with Matplotlib

Matplotlib is the standard Python library for publication-quality static charts. A clean, well-labelled chart requires deliberate attention to axes, labels, colours, and layout.

The following code produces the yield curve chart used in the structured products case study:

```
import matplotlib
matplotlib.use("Agg")  # non-interactive backend
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(T, rate, "o-", color="#E74C3C", lw=1.8,
        ms=5, label="Pillar rate (raw)")
ax.plot(T, zc_d, "s-", color="#2E86AB", lw=1.8,
        ms=5, label="ZC discrete")
ax.plot(T, zc_c, "^-", color="#27AE60", lw=1.8,
        ms=5, label="ZC continuous")

ax.set_xlabel("Tenor", fontsize=12,
              fontweight="bold")
ax.set_ylabel("Rate (%)", fontsize=12,
              fontweight="bold")
ax.set_title("Yield Curve: Pillar vs ZC Rates",
             fontsize=14, fontweight="bold")
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

plt.tight_layout()
```

```
plt.savefig("plots/yield_curve.png", dpi=150,
            bbox_inches="tight")
```

The code above produces the chart shown in Figure 1. A second example — the payoff diagram for the Energetic PEA structured product — demonstrates how a clean visualisation communicates product mechanics more effectively than a table of numbers:

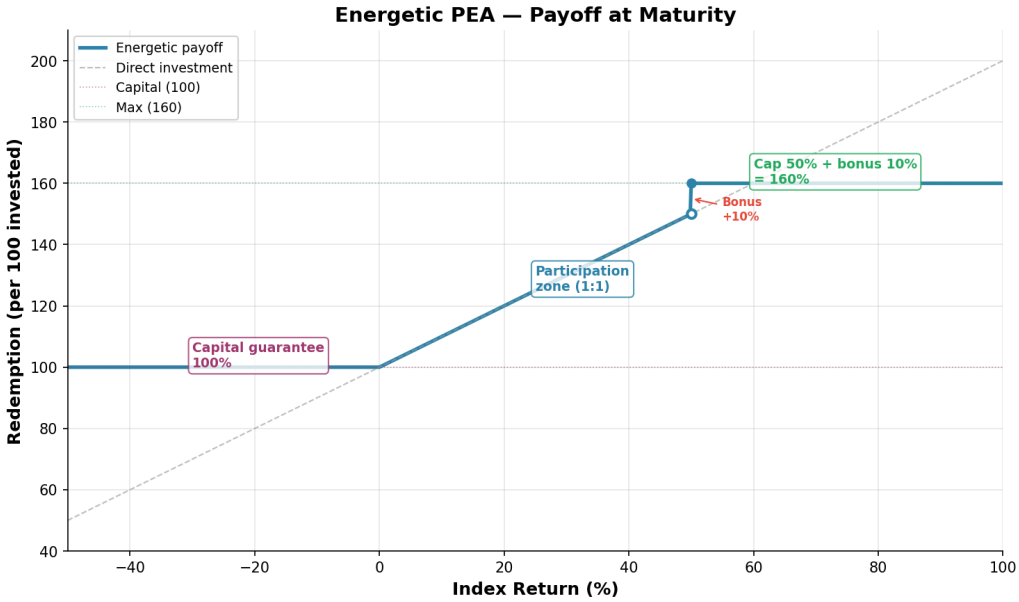

Figure 2: Payoff diagram for the Energetic PEA product, generated by the pipeline. The chart shows the capital guarantee floor at 100, linear participation between 0% and 50%, the discontinuity at 50% (bonus of 10 percentage points), and the cap at 160. This single figure replaces several paragraphs of textual description.

Key practices:

- **`matplotlib.use("Agg")`** selects a non-interactive backend, which is essential for headless environments (servers, CI pipelines) where no display is available.
- **`bbox_inches="tight"`** removes excess whitespace around the figure.
- **`dpi=150`** produces high-resolution output suitable for print and PDF embedding.
- **Removing top and right spines** produces a cleaner, more modern appearance.

> **Note** Always call `plt.savefig()` *before* `plt.show()`. Calling `show()` first clears the figure from memory, and the subsequent `savefig()` will produce a blank image.

4.2 Interactive Plots with Plotly

For exploratory analysis, interactive 3D visualizations allow the analyst to rotate, zoom, and hover over data points. The volatility surface — which maps implied volatility as a function of strike and maturity — is a natural candidate.

```
import plotly.graph_objects as go

fig = go.Figure(data=[go.Surface(
```

```
    z=vol_matrix,
    x=strikes,
    y=maturities,
    colorscale="Blues",
)])

fig.update_layout(
    title="Implied Volatility Surface",
    scene=dict(
        xaxis_title="Strike (%)",
        yaxis_title="Maturity (Y)",
        zaxis_title="Implied Vol (%)",
    ),
)

fig.write_html("plots/vol_surface_interactive.html")
```

The static version of this surface, produced by Matplotlib, is shown in Figure 3. The interactive HTML version allows the viewer to rotate the surface and hover over individual data points.

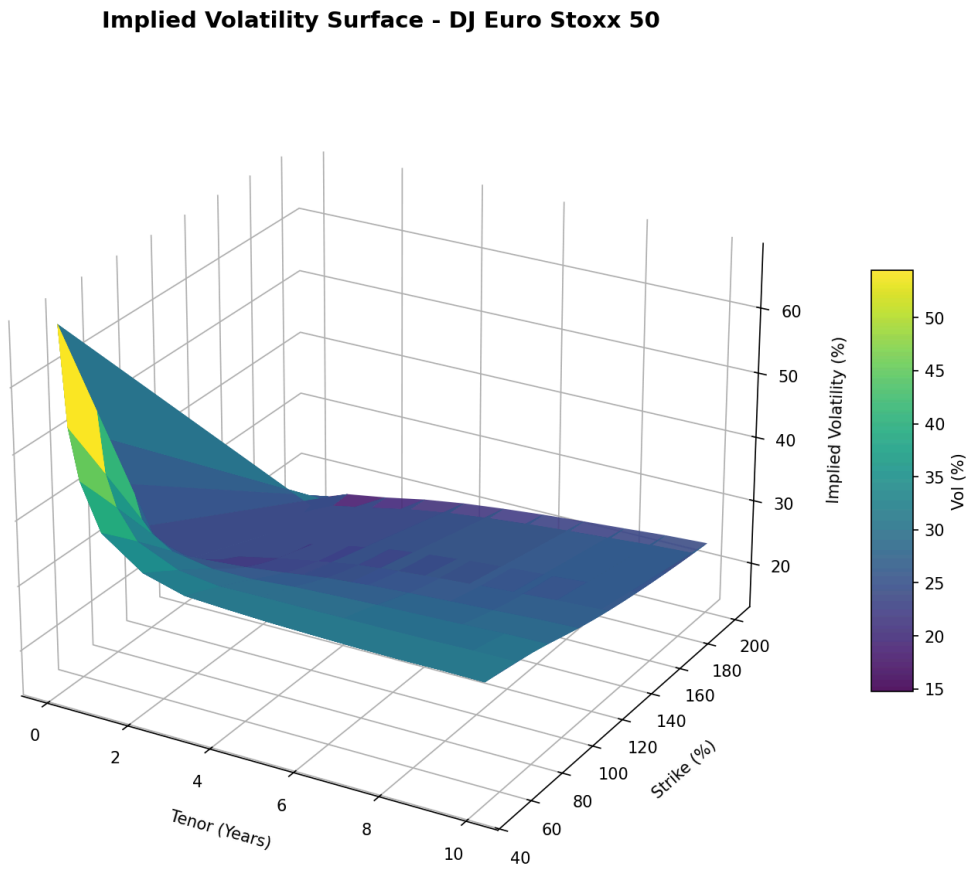

Figure 3: Implied volatility surface for the DJ Euro Stoxx 50, extracted from the case study pricer. The negative skew — higher implied volatility at low strikes — is characteristic of equity index options and reflects the market's pricing of downside tail risk. Strike is expressed as a percentage of spot; maturity ranges from 1 month to 10 years.

4.3 Typst for Professional Documents

Typst is a modern typesetting system designed for technical documents. It compiles plain-text source files into PDF with mathematical notation, code highlighting, and precise layout control — similar to LaTeX, but with a simpler syntax and faster compilation.

These lecture notes are themselves written in Typst. The key advantage for quantitative workflows is that Typst documents are **code** — they live in the repository alongside the Python scripts and can be compiled automatically by the orchestrator.

```
typst compile presentation.typ
```

This command converts the source file to PDF in under a second. By including this call in `main.py`, the report is always regenerated alongside the data, eliminating the risk of serving a stale document.

4.4 Excel as Data Source, Not Computation Engine

A recurring theme in this course is the transition from spreadsheet-based workflows to code-based pipelines. This is not a matter of preference — it is a matter of engineering capability.

Manual Excel Stripping	**Python Bootstrapping**
Drag formulas down cells by hand	Automated loop over all pillars
Minutes of manual interaction per run	Milliseconds of automated execution
Extending from 10Y to 50Y requires manual cell insertion	Changing a single loop bound
`#REF!` errors are opaque and difficult to trace	Stack traces pinpoint the exact line of failure
No version control — which cell changed between yesterday and today?	Full `git diff` history of every change to every formula
Cannot be included in a pipeline	Called by the orchestrator as one step among many

In a modern quantitative engineering workflow, the Excel file serves as a **data source** — a container of market quotes to be read by `xlrd` or `openpyxl`. The computation is performed entirely in Python, where it can be tested, versioned, and automated.

💡 **The role of Excel**

Excel is a data container, not a computation engine. The pricing logic belongs in version-controlled Python scripts that can be reviewed, tested, and executed automatically.

Key Takeaways

1. Use `matplotlib.use("Agg")` for headless environments; always call `savefig()` before `show()`.
2. Plotly HTML files enable interactive exploration of 3D surfaces — useful for presentations and client meetings.
3. Typst documents are code: they live in the repository and compile to PDF in the pipeline automatically.
4. Excel is a data container, not a computation engine — pricing logic belongs in versioned, testable Python scripts.

4.5 Prompt Quality: From Vague to Precise

The prompts in this chapter concern the *presentation layer*: charts, interactive visualisations, and typeset reports. The pattern is the same — context, constraints, output, verification — but the constraints are now aesthetic and format-related rather than mathematical.

4.5.1 Prompt set 10: Creating a publication-quality chart

✗ Bad prompt

"Plot the yield curve."

Which data? What axes? Which series? What file format? The agent will produce a default Matplotlib plot with no title, no legend, no axis labels, and low resolution — unusable in a professional report.

Average prompt

"Plot the three yield curve series (pillar rates, ZC discrete, ZC continuous) on a single chart. Save it as `plots/yield_curve.png`."

The series and the output path are specified. But the prompt says nothing about resolution, axis labels, colours, legend placement, or spine removal. The chart will be functional but will look like a homework plot, not a professional deliverable.

✓ Good prompt

"Create a publication-quality chart of the yield curve. Plot three series: pillar rates (red, circle markers), ZC discrete (blue, square markers), ZC continuous (green, triangle markers). X-axis: 'Tenor (years)', Y-axis: 'Rate (%)'. Add a title 'Yield Curve: Pillar vs ZC Rates'. Remove the top and right spines for a clean look. Use a subtle grid (alpha 0.3). Save as `plots/yield_curve.png` at 150 dpi with `bbox_inches='tight'`. Use `matplotlib.use('Agg')` so the script runs in headless environments. Call `savefig()` before `show()`."

Every visual decision is explicit: colours, markers, labels, spine removal, grid opacity, resolution, backend, and the save-before-show ordering. The agent produces a chart that is ready for a client presentation without manual touch-up.

4.5.2 Prompt set 11: Building an interactive visualisation

✗ Bad prompt

"Show the vol surface."

Show it how? As a static image? An interactive 3D plot? From which data source? The agent has no way to make a reasonable choice.

Average prompt

"Create a 3D plot of the implied volatility surface from the case study data. Use Plotly so it's interactive."

The library and the interactivity requirement are clear. But the prompt does not specify the data source (which sheet, which file), the axis labels, the colour scale, or the output format. The agent will produce a plot, but the axes may be mislabelled or the data may come from the wrong sheet.

✓ Good prompt

"Build an interactive 3D volatility surface using Plotly. Read implied volatility data from the sheet 'Vol implicite' in `pricer-energy-3.xls`*. X-axis: strike as percentage of spot. Y-axis: maturity in years. Z-axis: implied volatility in percent. Use the 'Blues' colour scale. Save the output as a self-contained HTML file at* `plots/vol_surface_interactive.html`*. Also generate a static PNG version with Matplotlib (*`plots/vol_surface.png`*) for embedding in the Typst report. The static version should use the same axis labels and a coherent colour scheme."*

The prompt specifies both output formats (interactive HTML + static PNG), ensuring the visualisation works in both presentations (where interactivity matters) and printed reports (where a static image is required). Axis semantics and data source are explicit.

4.5.3 Prompt set 12: Generating an automated report

✗ Bad prompt

"Make a PDF report."

A report about what? Using which tool? Containing which charts? The agent will likely generate a simple Markdown-to-PDF script, ignoring the Typst-based pipeline established in previous chapters.

Average prompt

"Write a Typst document that includes the yield curve chart and the pricing results. Compile it to PDF."

The tool is correct and the charts are named. But the prompt does not specify the document title, the section structure, which figures to include, or how the compilation should integrate with `main.py`.

✓ Good prompt

"Create a Typst source file `presentation.typ` that serves as the client-facing deliverable for the Energetic PEA case study. Structure: (1) title page with product name, date, and author; (2) yield curve section embedding `plots/yield_curve.png` with a caption explaining the three rate series; (3) payoff section embedding `plots/payoff_energetic.png`; (4) pricing results table with the theoretical price and component decomposition; (5) sensitivity analysis charts. Add the `typst compile presentation.typ` command as the final step in `main.py` so that the PDF is regenerated every time the pipeline runs. This way, the report is never stale."

The document structure is fully specified: sections, figures, tables, and the integration with the orchestrator. The key insight — placing compilation at the end of `main.py` — ensures the report is always current, which is the central argument of this chapter.

4.6 Questions

4.6.1 Open Questions

Question O4.1 — A structuring team uses an Excel pricer with embedded VBA macros to compute margin on a new product. The process takes approximately 15 minutes per run (including manual chart updates). Using the concepts from this chapter, design a Python-based replacement workflow. Specify which libraries you would use for data reading, computation, visualisation, and report generation. Estimate the time savings.

Question O4.2 — Explain why removing the top and right spines from a Matplotlib chart, while purely aesthetic, can be considered an engineering best practice. Your answer should address the concept of the *data-to-ink ratio*.

Question O4.3 — A colleague produces a presentation by manually copying Matplotlib charts into PowerPoint and annotating them by hand. When the input data changes, the entire process must be repeated. Describe how integrating Typst (or a similar programmatic typesetting tool) into the orchestrator eliminates this problem.

4.6.2 Multiple Choice Questions

Question 16 — In Matplotlib, `matplotlib.use("Agg")` is called before any plotting. Why?

A. It makes plots render faster
B. It selects a non-interactive backend, enabling plot generation in headless environments (servers, CI pipelines) where no display is available
C. It changes the default colour scheme
D. It is required by `uv`
E. It enables interactive zooming

Question 17 — The orchestrator calls `typst compile presentation.typ` as its final step. What does this guarantee?

A. The PDF is uploaded to a server
B. The presentation PDF always reflects the latest data and plots, without manual intervention
C. Typst runs the Python scripts
D. The PDF is encrypted
E. Compilation takes several minutes

Question 18 — A colleague extends the yield curve from 10 years to 30 years. In Python, this requires:

A. Rebuilding the entire project
B. Changing the loop bound in the bootstrapping script and re-running the pipeline
C. Manually inserting 20 new rows in an Excel spreadsheet
D. Rewriting the `pyproject.toml`
E. Installing a new package

Question 19 — Plotly outputs an HTML file. What is the advantage over a static PNG?

A. Smaller file size
B. HTML files can contain Python code
C. PNG files cannot be embedded in PDFs
D. The viewer can rotate, zoom, and hover over data points — useful for exploring 3D surfaces
E. Plotly is faster than Matplotlib

Question 20 — Why is `plt.savefig()` called before `plt.show()`?

A. It is a Python convention with no technical reason
B. `plt.show()` clears the figure from memory; calling `savefig()` afterward would produce a blank file
C. `savefig()` does not work after `show()` due to a Matplotlib bug
D. The order does not matter
E. `show()` compresses the image quality

Conclusion

These notes have presented four disciplines that, taken together, constitute what we call *software engineering for derivative pricing*.

Environment management (Chapter 1) ensures that a project is reproducible across machines and across time. A `pyproject.toml` declares the project's dependencies; `uv.lock` pins their exact versions; `uv run` guarantees execution inside a controlled environment. The virtual environment is disposable — the lockfile is the source of truth.

Project architecture (Chapter 2) ensures that a project is legible and automatable. Separating input (`data/`), logic (`scripts/`), and output (`plots/`, `logs/`) makes the project's structure self-documenting. The orchestrator pattern consolidates the entire pipeline into a single command: `uv run python main.py`.

Verification methodology (Chapter 3) ensures that results are trustworthy. AI code generators produce functional scripts in seconds, but "runs without errors" is not a certificate of correctness. The engineer's role shifts from writing code to specifying problems precisely and auditing output rigorously — through manual spot-checks and domain knowledge.

Professional reporting (Chapter 4) ensures that results are communicable. Publication-quality charts, interactive visualisations, and typeset documents are not cosmetic — they are the medium through which quantitative work is evaluated and acted upon.

4.7 Project Readiness Checklist

The following checklist can be applied to any quantitative project. A project that satisfies every item is ready for professional delivery.

#	Criterion
1	A `pyproject.toml` declares all dependencies.
2	A `uv.lock` file is committed to version control.
3	`uv run python main.py` executes the full pipeline without manual intervention.
4	The `data/` directory is read-only — no script modifies source files.
5	Deleting `plots/` and `logs/` has no impact on the project's integrity.
6	At least one algorithmic step has been manually verified (e.g., computing one or two discount factors by hand).
7	The output has been audited against domain knowledge (e.g., the yield curve rate hierarchy is consistent with financial theory).
8	Charts are labelled, high-resolution, and saved programmatically — not manually exported.
9	The project compiles its own reports (PDF) as the final pipeline step.
10	A README documents the project structure, the data sources, and the commands required to reproduce the results.

4.8 The Engineering Mindset

The tools presented in these notes — `uv`, Typst, Matplotlib, Plotly, the orchestrator pattern — will evolve. Package managers will be replaced by faster ones; typesetting systems will gain new features; AI coding agents will become more capable.

What will not change is the *methodology*:

- **Specify precisely** before generating.
- **Verify manually** before trusting.
- **Automate completely** before delivering.
- **Document visibly** before archiving.

A quantitative engineer who internalises these principles can adopt any new tool in an afternoon. An engineer who memorises tools without understanding principles will be obsolete with each technological cycle.

The structured product was the vehicle. The engineering was the destination.

Appendix A — Answer Key

Q	Ans	Rationale
1	D	`uv sync` reads the lockfile and installs exact pinned versions.
2	B	`pyproject.toml` declares ranges; `uv.lock` pins exact versions.
3	E	`uv run` ensures env exists and is synchronised before executing.
4	B	`.venv/` is disposable; `uv sync` recreates it from the lockfile.
5	C	Global `pip install` creates untracked, unreproducible dependencies.
6	C	`data/` holds immutable source files, never modified by scripts.
7	A	`sys.executable` ensures the script runs inside the same virtual environment.
8	B	The orchestrator aborts on failure to prevent downstream scripts from running on missing data.
9	C	`plots/` is generated output; `main.py` regenerates it from scratch.
10	A	Typst compilation as the final step keeps the PDF synchronised with the latest data.
11	D	Running without errors does not prove correctness; manual verification is required.
12	C	Matching the first non-trivial iteration validates the loop logic.
13	D	The coupon stripping effect causes ZC rates to exceed par rates on an upward-sloping curve.
14	B	The lockfile pins `scipy==1.17.1` regardless of newer releases.
15	E	An incorrect day-count convention produces systematically biased but plausible-looking results.
16	B	`Agg` is a non-interactive backend for headless environments.
17	B	Automated PDF compilation ensures the deliverable is always current.
18	B	Extending the curve in Python requires only changing the loop bound.
19	D	Plotly HTML allows rotation, zoom, and hover — useful for 3D exploration.
20	B	`plt.show()` clears the figure; `savefig()` must be called first.

Appendix B — Tools & References

Tool	Role	URL
`uv`	Package & environment manager	https://docs.astral.sh/uv/
Python	Computation language	https://www.python.org/
Typst	Document typesetting	https://typst.app/
Matplotlib	Static plotting	https://matplotlib.org/
Plotly	Interactive plotting	https://plotly.com/python/
SciPy	Scientific computation	https://scipy.org/
`xlrd`	Reading legacy `.xls` files	https://pypi.org/project/xlrd/
`openpyxl`	Reading `.xlsx` files	https://openpyxl.readthedocs.io/
Git	Version control	https://git-scm.com/

4.8.1 Recommended Reading

- Astral documentation: *Getting Started with uv* — https://docs.astral.sh/uv/getting-started/
- Python Packaging User Guide — https://packaging.python.org/
- Typst documentation — https://typst.app/docs/

4.8.2 References

Barchechath, J. (2026). *Equity structured products: Case study materials* [Course materials]. Structured Products, MSc Financial Markets and Investments, SKEMA Business School. http://structuredproducts.free.fr